



GitHub Trending Top 10

每日热门开源项目深度解读

2026-08-19

今日榜单

1	harry0703/MoneyPrinterTurbo	Python	利用 AI 大模型和自动化 workflow，根据主题或关键词一键生成高清短视频。Generate HD short vi...	2天
2	chaitanyagiri/munder-difflin	TypeScript	local multi-agent harness	NEW
3	akitaonrails/ai-memory	Rust	Solution for long term memory for agent coding CLIs and to facilitate handoff be...	2天
4	volcengine/OpenViking	Python	Self-evolving Context Database for AI Agents. Unify Agent Memory, Knowledge RAG ...	NEW
5	mukul975/Anthropic-Cybersecurity-Skills	Python	817 structured cybersecurity skills for AI agents • Mapped to 6 frameworks: MITR...	2天
6	public-apis/public-apis	Python	A collective list of free APIs	NEW
7	basecamp/omarchy	Shell	Beautiful, Modern & Opinionated Linux	NEW
8	agalwood/Motrix	TypeScript	A full-featured download manager.	NEW
9	NawfalMotii79/PLFM_RADAR	PLSQL	Open-source, low-cost 10.5 GHz PLFM phased array RADAR system	NEW
10	jundot/omlx	Python	LLM inference server with continuous batching & SSD caching for Apple Silicon — ...	2天

#1

Python

连续 2 天

MoneyPrinterTurbo

利用 AI 大模型和自动化工作流，根据主题或关键词一键生成高清短视频。Generate HD short videos from a topic or keyword with an automated AI workflow.

Stars **109,550** Forks **16,628** Today **+2,304**

<https://github.com/harry0703/MoneyPrinterTurbo>

项目简介：MoneyPrinterTurbo 是一款基于 AI 大模型的一站式短视频自动生成工具，用户只需输入主题或关键词，即可自动完成脚本撰写、素材匹配、字幕生成、配音和背景音乐合成，最终输出高清短视频。它本质上把“短视频内容生产”从人工剪辑的繁琐流程，压缩成了一条全自动流水线。

核心功能：一是智能脚本生成，基于大模型自动撰写视频文案并提炼素材搜索关键词；二是素材自动匹配，根据脚本语义从图库/视频库中挑选贴合画面的素材；三是全自动合成，集成字幕渲染、TTS 配音和背景音乐混音，直接输出成品视频；四是双模式操作，提供 WebUI 图形界面和 API 接口，既适合个人使用也便于二次开发集成。

适用场景：最典型的用户是自媒体运营者、短视频营销人员和内容创业者，用于批量产出口播类、科普类或资讯类视频。也适合需要快速生成视频素材的团队，比如电商产品介绍、新闻快讯播报，以及想用 AI 做内容实验的开发者。

技术亮点：项目将多个 AI 能力（LLM 文案、TTS 语音、视频合成）编排成一条自动化工作流，且模块化设计让各环节可替换——比如可自由切换不同的大模型或 TTS 引擎。此外，它同时提供 WebUI 和 REST API，降低了使用门槛，也方便开发者将其嵌入到更大的内容生产系统中。

上手建议：如果你是使用者，直接下载最新 Release 的安装包，按文档配置好大模型 API Key（支持 OpenAI、Kimi、DeepSeek 等）即可在 WebUI 中体验。如果你是开发者，建议先阅读 README 中的架构说明和 API 文档，从调用 `/api/v1/videos` 接口开始理解整个生成流程，再逐步深入各模块的源码进行定制或贡献。

#2

TypeScript

NEW

munder-difflin

local multi-agent harness

Stars **2,397** Forks **280** Today **+306**<https://github.com/chaitanyagiri/munder-difflin>

项目简介：Munder Difflin 是一个开源的多智能体协调框架（multi-agent harness），它把你日常使用的终端编码 CLI（如 Claude Code、Codex、Grok 等）包装成一个个独立的 AI “员工”，并让它们在一个虚拟办公室中协同工作。它解决的核心问题是：如何让多个不同模型的 AI 代理在同一台机器上形成有组织、有记忆、可调度的团队，而不是各自为战。

核心功能：1) **多模型接入**——原生支持 Claude、Gemini、OpenAI、Grok、Qwen 等 10 余种终端 CLI，自带 API Key 或本地模型均可；2) **虚拟办公室**——用 Pixi.js 渲染 2D 办公室，每个代理是一个头像，工作时走动到工位，消息以信封动画传递；3) **智能路由与记忆**——内置 GOD 代理（Michael）负责任务分配和仲裁，所有代理共享毫秒级语义记忆层，跨会话记住上下文；4) **真实终端交互**——通过 node-pty 运行真实进程，xterm.js 渲染，保证与原生 CLI 行为完全一致。

适用场景：适合需要同时使用多个 AI 编码工具的高级开发者、AI 研究者和自动化爱好者。典型场景包括：让 Claude 写代码、Codex 做审查、Grok 查资料，由一个调度代理统筹；或者在你离开时让代理团队持续处理积压的编码任务。如果你已经为多个 AI CLI 付费，这个工具能让你“榨干”它们的订阅额度。

技术亮点：最值得关注的是它的**分层架构**——UI（Electron + React）与代理运行时（node-pty 进程）完全解耦，每个代理是真实操作系统进程而非 API 模拟调用，保证了极高的真实性。其次是**混合记忆系统**：Markdown 为主存储 + 语义索引辅助检索，兼顾了人类可读性和机器检索速度。另外，用 Pixi.js 做实时动画层来可视化代理状态，在同类工具中相当新颖。

上手建议：如果你是用户，先安装依赖的 CLI（至少一个，如 Claude Code），然后从 GitHub Releases 下载对应平台的安装包即可运行。如果你是贡献者，建议先阅读 CONTRIBUTING.md 和 ARCHITECTURE.md（如果存在），从 GitHub Issues 中的 good first issue 标签入手，项目明确欢迎 PR。项目目前标注为“working prototype”，意味着核心功能可用但 API 可能变动，参与前最好先加入 Discord 了解最新开发方向。

#3

Rust

连续 2 天

ai-memory

Solution for long term memory for agent coding CLIs and to facilitate handoff between different agent vendors

Stars **3,009** Forks **251** Today **+648**

<https://github.com/akitaonrails/ai-memory>

项目简介：ai-memory 是一个为 AI 编程代理提供长期记忆的 Rust 命令行工具。它解决了不同 AI 编码代理（如 Claude Code、Codex 等）之间切换时丢失上下文的问题，让你在中断任务后换一个代理工具，依然能无缝接续工作，无需重新解释架构或已尝试过的思路。

核心功能：

- **跨代理记忆持久化：**通过 MCP 配置和生命周期钩子，自动捕获每个代理的会话上下文、决策过程和未解决问题
- **会话交接：**支持在不同代理工具之间无缝切换，新代理能读取之前代理的完整工作记忆
- **多平台支持：**覆盖 Linux、macOS、Windows（WSL2 和原生）、以及十余种主流 AI 编码代理工具
- **托管工作流：**通过 ai-memory run 命令实现跨工具的会话连续性，不改变原有的启动方式

适用场景：适合重度使用 AI 编码代理的开发者，尤其是那些会在多个工具之间切换（比如白天用 Claude Code，晚上用 Codex）或者经常中断长任务的用户。也适合团队协作场景，当不同成员使用不同 AI 代理时，确保项目上下文能共享。

技术亮点：采用 Rust 实现，性能出色且便于分发为原生二进制。架构上通过 MCP 协议和生命周期钩子实现代理无关的上下文捕获，设计精妙。对不同代理的适配做了精细处理（如支持 Devin 的 PostCompaction 事件、Command Code 的四个生命周期事件），显示出对每个生态的深入理解。会话感知的 MCP 桥接设计也体现了对隔离性和安全性的考量。

上手建议：从 README 的支持矩阵入手，找到你正在使用的代理工具，按照对应文档完成安装和 MCP 配置。初次使用建议先用一个简单项目测试，体验跨代理切换的流程。如果想贡献代码，可以从 Rust 代码库的 docs/ 目录开始了解架构设计，再关注 GitHub Issues 中标记为 good first issue 的任务。

#4

Python

NEW

OpenViking

Self-evolving Context Database for AI Agents. Unify Agent Memory, Knowledge RAG and Skills.

Stars **29,785** Forks **2,319** Today **+213**

<https://github.com/volcengine/OpenViking>

项目简介：OpenViking 是字节跳动开源的一个面向 AI Agent 的“上下文数据库”，把智能体的记忆、知识库和技能统一抽象成一个虚拟文件系统（viking:// 协议）。它解决了当前 RAG 系统“黑盒向量检索、上下文碎片化、Token 消耗高”的痛点，让 Agent 能像人类操作文件系统一样管理自己的上下文。

核心功能：1) 三层内容分级（L0 摘要 / L1 概览 / L2 详情），按需加载以节省 Token；2) 目录递归检索，先定位高相关目录再逐层下钻，保留上下文完整性；3) 检索轨迹可视化，每次查询路径可回放调试；4) 会话自动沉淀为长期记忆，异步提取用户偏好与经验；5) 统一 URI 管理记忆、资源与技能，支持确定性定位。

适用场景：适合构建复杂 AI Agent 的团队，尤其是需要长期记忆、多轮对话、私有知识库管理的场景，如智能客服、编程助手、个人知识管家等。对 Token 成本敏感的生产级 Agent 应用尤其有价值。

技术亮点：核心设计是用“文件系统隐喻”替代传统的纯向量检索——Agent 用 ls、tree、find 浏览自己的上下文，而非黑盒查询。分级存储 + 按需加载显著降低推理成本，而目录递归检索保证了返回结果带着完整上下文，避免碎片化。检索轨迹可观测性也是工程实践上的加分项。

上手建议：建议先体验官网的 Live Demo（OpenViking Studio），直观感受文件系统式上下文浏览。然后阅读官方文档的 Getting Started 和 Architecture 章节，理解核心概念。想深入贡献的话，可以从 GitHub Issues 入手，项目 Python 技术栈，AGPLv3 协议，注意商业使用需谨慎。

#5

Python

连续 2 天

Anthropic-Cybersecurity-Skills

817 structured cybersecurity skills for AI agents • Mapped to 6 frameworks: MITRE ATT&CK, NIST CSF 2.0, MITRE ATLAS, D3FEND, NIST AI RMF & MITRE F3 (Fight Fraud) • agentskills.io standard • Works with Claude Code, GitHub Copilot, Codex CLI, Cursor, Gemini CLI & 20+ platforms • 29 security domains • Apache 2.0

Stars **29,505** Forks **3,510** Today **+730**

<https://github.com/mukul975/Anthropic-Cybersecurity-Skills>

项目简介

这是一个为AI智能体（如Claude Code、Copilot等）提供网络安全专业能力的开源技能库，包含817个结构化的安全操作技能，覆盖29个安全领域。它解决了AI助手缺乏深度安全知识的问题，让普通开发者也能获得接近资深安全分析师的操作指导。

核心功能

- 技能库覆盖6大行业框架（MITRE ATT&CK、NIST CSF 2.0、ATLAS、D3FEND、AI RMF、F3），每个技能都映射到相关框架
- 兼容26+主流AI编程平台（Claude Code、Cursor、Gemini CLI等），即插即用
- 技能内容涵盖从数字取证、红队操作到AI安全、反欺诈等29个细分领域
- 遵循agentskills.io开放标准，技能格式统一，易于扩展和复用

适用场景

- 安全分析师用AI辅助进行事件响应、恶意软件分析或云安全审计
- 红队/渗透测试人员借助AI快速完成侦察、漏洞利用等步骤
- 开发者需要在代码中集成安全检测能力，或为AI应用增加安全防护
- 安全教育和培训场景，作为AI驱动的教学工具

技术亮点

- 采用“技能即代码”的理念，将专家经验编码为结构化指令，AI可直接执行
- 框架映射策略很聪明：每个技能只关联相关框架（如取证技能带ATT&CK+CSF，AI安全技能额外加ATLAS+AI RMF），避免冗余
- 作为社区项目（非Anthropic官方），Apache 2.0许可降低了商业使用门槛，同时明确标注了合规使用边界

上手建议

- 先浏览README中的技能分类表，找到你关注的安全领域（如云安全、取证）
- 克隆仓库后，选择你常用的AI平台（如Claude Code），按quick-start指引将技能目录指向你的agent
- 想贡献的话，从阅读CONTRIBUTING.md开始，参考现有技能格式提交新技能，注意遵循agentskills.io标准
- 注意使用限制：攻击性技能仅用于授权测试，务必遵守法律和道德规范

#6

Python

NEW

public-apis

A collective list of free APIs

Stars **465,279** Forks **51,371** Today **+1,005**

<https://github.com/public-apis/public-apis>

项目简介：这是一个社区维护的免费公共 API 集合列表，堪称开发者的“API 百宝箱”。它解决了开发者寻找可靠、免费 API 时信息分散、质量参差不齐的痛点，将数百个经过验证的公共接口按类别整理在一起。

核心功能：① 按领域分类整理数百个免费 API（涵盖金融、天气、地理、娱乐等）；② 每个 API 都附有简要说明和官方链接，方便快速筛选；③ 社区驱动的持续更新机制，确保列表的时效性和质量；④ 顶部集成了 APILayer 的商业 API 推荐，为有更高需求的用户提供升级路径。

适用场景：前端开发者做 demo 或 MVP 时快速寻找数据源；学生和爱好者学习 API 调用和前后端交互；产品经理和技术调研人员评估可用的第三方服务；以及任何需要快速获取公开数据（如汇率、天气、笑话、菜谱）的个人项目。

技术亮点：项目本身技术含量不高（本质是 Markdown 列表），但其价值在于“众包治理”模式——通过严格的贡献指南、分类标签和社区审查，将零散的 API 信息转化为结构化知识库。46 万+ 星标也印证了这种“简单但极其实用”模式的成功。

上手建议：直接浏览 README 中的分类目录，找到感兴趣的 API 后点击链接查看文档即可开始调用。如果想贡献，先阅读 CONTRIBUTING 指南，按照格式要求提交新的 API 条目（需附上说明、链接和分类），等待维护者审核合并即可。

#7

Shell

NEW

omarchy

Beautiful, Modern & Opinionated Linux

Stars **26,624** Forks **2,707** Today **+356**

<https://github.com/basecamp/omarchy>

项目简介：Omarchy 是由 Rails 创始人 DHH 主导的一个 Linux 发行版项目，主打“美观、现代、有主见”的开箱即用体验。它本质上是一套深度定制的 Linux 系统配置方案，用 Shell 脚本封装了从桌面环境到开发工具链的完整生态，解决的是“Linux 太折腾、默认体验不统一”的痛点。

核心功能：1) 预配置的现代化桌面环境，包括顶部栏、主题、热键和统一剪贴板历史；2) 内置完整的开发工具链，从 Neovim、终端到 AI 辅助工具一应俱全；3) 提供 Omarchy CLI 命令行工具，用于系统管理和日常操作；4) 丰富的应用覆盖，从浏览器、游戏到 Windows 虚拟机，几乎涵盖日常所有需求；5) 详尽的 52 章手册，将系统配置和用法文档化，便于学习和复现。

适用场景：适合两类人：一是想从 macOS/Windows 切换到 Linux 的开发者，希望获得类似“苹果式”的精致开箱体验；二是愿意接受“有主见”默认配置、不想花大量时间折腾桌面环境的 Linux 用户。对于想学习如何系统化配置 Linux 的爱好者，这份手册也是极佳的学习材料。

技术亮点：项目以 Shell 脚本作为核心实现，而非传统的打包方式，意味着整个系统配置是“可读、可审计、可修改”的。最值得关注的是其“文档即权威”的理念——手册是系统的唯一事实来源，代码与文档高度同步，这在开源系统中相当罕见。此外，它集成了现代 AI 工具和硬件认证等前沿功能，体现了对开发者体验的深度思考。

上手建议：如果你是使用者，建议先阅读 `manual/01-welcome-to-omarchy.md` 和 `02-getting-started.md`，了解安装和基本概念；如果你是贡献者，可以从阅读 `manual/` 目录下手，先熟悉系统架构和设计哲学，再通过 GitHub Issues 或 PR 提交改进。由于项目仍处于快速迭代期，建议在虚拟机中先试运行。

#8

TypeScript

NEW

Motrix

A full-featured download manager.

Stars **53,942** Forks **4,958** Today **+609**

<https://github.com/agalwood/Motrix>

项目简介：Motrix 是一款现代化的全功能下载管理器，支持 HTTP、FTP、BitTorrent 和磁力链接等多种下载协议。它旨在解决传统下载工具界面复杂、功能分散的问题，提供一个简洁易用但功能强大的统一下载入口。

核心功能：多协议下载支持（HTTP/FTP/BT/磁力链）、基于 SQLite 的会话恢复机制、可自定义的仪表盘与速度限制配置、QuickJS 插件沙箱系统，以及浏览器扩展一键接管下载。此外，它还提供 Docker 无头服务器模式，方便部署在 NAS 或家庭服务器上。

适用场景：日常用户需要管理多种类型下载任务时；开发者或 AI 代理需要通过命令行工具（@motrix/cli）自动化下载流程时；NAS 用户希望可在无桌面环境中运行下载服务时。浏览器扩展特别适合经常从 Chrome/Firefox 下载文件的用户。

技术亮点：v2 版本采用 Electron + React + TypeScript 重构，核心下载引擎与 UI 完全解耦，通过开放的 MDXP 协议（基于 JSON-RPC 2.0）进行通信。插件运行在隔离的 QuickJS 沙箱中，配合细粒度权限控制，安全性和可扩展性俱佳。

上手建议：普通用户可直接从 GitHub Releases 下载 v2 beta 版本体验，但需注意备份 v1 数据（迁移尚未验证）。开发者可从 @motrix/mdxp 协议库或 Plugin SDK 入手，了解扩展开发；想要贡献核心代码的，建议先阅读架构文档和 beta 发布说明，了解当前重构进度。

#9

PLSQL

NEW

PLFM_RADAR

Open-source, low-cost 10.5 GHz PLFM phased array RADAR system

Stars **24,479** Forks **5,614** Today **+192**

https://github.com/NawfalMotii79/PLFM_RADAR

项目简介：AERIS-10 是一个开源的低成本 10.5 GHz 相控阵雷达系统，采用脉冲线性调频（LFM）调制技术。该项目旨在将雷达技术平民化，让研究人员、无人机开发者和 SDR 爱好者能够以可负担的成本接触和实验相控阵雷达技术，而非仅停留在理论层面。

核心功能：系统提供 3km 和 20km 两个版本（Nexus 与 Extended），支持 $\pm 45^\circ$ 全电子波束扫描。内置 FPGA 完成脉冲压缩、多普勒 FFT、MTI 和 CFAR 等完整信号处理链，配合 Python GUI 实现地图集成与可视化，同时支持 GPS/IMU 实时姿态校正。

适用场景：高校雷达课程与科研实验、无人机避障与目标跟踪系统开发、业余无线电爱好者的高级 SDR 实验平台。对于初创公司而言，这也是一个快速原型验证的低成本起点。

技术亮点：全链路开源（硬件 CERN-OHL-P、软件 MIT）且模块化设计，射频前端采用 ADAR1000 移相器与 ADTR1107 收发芯片，配合 AD9523-1 时钟同步方案。特别值得注意的是其混合 AGC 环路（FPGA/STM32/GUI 三层协同），以及 32x16 波导阵列带来的 20km 探测能力，这在开源项目中极为罕见。

上手建议：建议从 README 中的系统框图开始理解整体架构，然后关注 8_Utils 目录下的设计文件。初学者可从 AERIS-10N（3km 版本）入手，先在仿真环境验证信号处理链，再逐步搭建硬件。贡献者可从 GitHub Issues 中的 Work in Progress 功能入手，项目明确欢迎 PR。

#10

Python

连续 2 天

omlx

LLM inference server with continuous batching & SSD caching for Apple Silicon — managed from the macOS menu bar

Stars **19,624** Forks **1,681** Today **+370**

<https://github.com/jundot/omlx>

项目简介：oMLX 是一个专为 Apple Silicon Mac 设计的 LLM 推理服务器，通过菜单栏应用即可管理本地大模型的加载、切换和推理。它解决了本地运行 LLM 时“便利性”与“控制力”不可兼得的痛点，让开发者既能像使用云服务一样轻松调用模型，又能精细控制内存占用和上下文长度。

核心功能：一是连续批处理（continuous batching），能高效处理并发请求，提升吞吐量；二是分层 KV 缓存，将热数据驻留内存、冷数据落盘 SSD，即使对话上下文变化也能复用历史缓存；三是菜单栏原生管理界面，支持一键启动/停止服务、下载模型、切换模型和查看实时状态；四是提供 CLI 和 MCP 支持，方便集成到 Claude Code 等外部工具链。

适用场景：主要面向在 Mac 上进行本地 AI 开发的工程师，尤其是使用 Claude Code、OpenCode、Cursor 等编码助手时希望数据不出本机的用户。也适合需要同时运行多个模型、频繁切换上下文窗口（如长文档分析）的研究者，以及希望用 Apple Shortcuts 自动化控制模型服务的效率爱好者。

技术亮点：其核心创新在于“跨上下文持久化 KV 缓存”——当对话上下文长度变化时，传统方案会丢弃全部缓存，而 oMLX 通过 SSD 冷存储保留历史 token 的 KV 状态，实现缓存复用，这在长会话场景下能大幅降低首 token 延迟。此外，项目为 GLM-5.2、MiniMax M3 等模型提供定制 Metal 内核，实测可将 prefill 速度提升约 30 倍（845 vs 29 tok/s），展现了针对 Apple Silicon 的深度优化能力。

上手建议：普通用户直接下载 DMG 安装包即可，图形界面引导清晰，无需配置环境。开发者若想从源码构建，需注意 macOS 15+、Python 3.11-3.13 和 Xcode 完整工具链（非仅 Command Line Tools），因为自定义内核编译依赖 Metal 编译器。建议先通过 `pip install -e .` 安装核心版本体验，再按需启用 MCP 或自定义内核，贡献代码可从 GitHub Issues 中标记“good first issue”的任务入手。

数据来源: github.com/trending

报告日期: 2026-08-19

由 DeepSeek AI 提供智能分析 | 自动生成